

Using the ad1 symbolic debugger for the PDP-1

This document describes the **ad1** symbolic debugger and how to use it.

This is version 1.1 and covers up through ad1 version 1.0; it will be updated as needed.

Edit date 1-Mar-2026 minor edits

What is ad1?

(Yet) Another Debugger for the PDP-1.

Why do we need another one?

While **DDT** was revolutionary for its time, its time has long past.

It was memory resident and consumed at least 25% of bank 0 memory.

It required tedious procedures to set up and use.

It was limited in the number of breakpoints it could set, one.

Its command structure was, well, a bit obscure.

To be fair, many of these issues were a result of the very limited memory available.

The features

One major difference from **DDT**, aside from the obvious added features, is that **ad1** is not memory resident in the pidp-1's memory. It is a standalone C program that interacts with the pidp-1 via a shared memory segment and hence does not need to be loaded nor does it consume any pidp-1 memory at all.

Features

- Uses symbol tables from **am1** to provide symbolic names for program locations and variables
- Uses listing files from **am1** to provide viewing of the source by address, symbol, or line number
- Can set up to 8 breakpoints, configurable to any number at build time
- Individual breakpoints can be enabled, disabled, or deleted
- Breakpoints can have a hit count and won't be raised until the count is reached
- Can set up to 8 watches, configurable to any number at build time
- Individual watches can be enabled, disabled, or deleted
- Watches can have a value to match on write, or match on any write
- Memory locations can be interrogated and set
- Has full access to the PC, PF, SS, TW, TA, MA, MB, IO, and AC registers
- Most of those can be both interrogated and set
- Can interrogate the test switches and sense switches
- Fully supports multi-bank operations
- Numeric values can be in base 2, 8, 10, and 16
- A numeric value can override the base setting by specifying its base as in 0177, 0d23, 0xFF, 0b11001

- Data can be viewed as numbers, ascii, or flex/concise
- Has a rich set of commands
- Has built-in help for commands

What goes on inside?

It is implemented in **C** using **Flex** and **Bison** for command parsing.

Central to its operation is the use of a shared memory segment that contains the full operating state of the pidp-1. This becomes available when the pidp-1 is started in *shared* mode via its configuration file. This does impose the restriction that **ad1** must be running on the same machine as the pidp-1 instance being debugged.

This is not only used for interrogating and setting the above registers and memory, it also allows control over starting, stopping, single-stepping and processing of breakpoints and watches.

Ad1 operates totally asynchronously, all operations can be done while the pidp-1 is running.

Breakpoints are implemented by a breakpoint table in shared memory. When breakpoints are active, an assignment to the PC register is checked to see if it is a breakpoint address and if so and its hit count is reached, a flag is set to indicate that to **ad1** and the pidp-1 halted.

Watchpoints are implemented similarly with a watch table in shared memory. When watches are active, the addresses have their contents checked to see if they have changed and optionally match a given value. If so a flag is set to indicate that to **ad1** and the pidp-1 halted. Both breakpoints and watches are checked in the pidp-1 at the end of every machine cycle.

Ad1 commands can then be given and when ready the program resumed.

No modification of program memory is done or needed, unlike the traditional **DDT** method of modifying instructions. This also means a breakpoint *can* be set on an instruction that is modified by the program, the breakpoint only depends upon the PC address.

Deleting a breakpoint just clears the associated entry in the breakpoint table.

However, as is the same for **DDT**, there is a restriction and for the same reason.

A breakpoint cannot be set on data, since that will not be addressed by the PC register.

Instead, a watch can be used since it only cares about the memory contents.

If a symbol table is given, it is parsed to build a map of symbol names to addresses and vice-versa. If an address has an associated symbol, the symbol will be displayed. If a symbol is given, it is used as an address.

Similarly, if a listing file is given, a map of addresses to line numbers is built. Lines from the file can be searched by address, symbol name, or line number and displayed.

If only a source file is available or the program was not assembled by **am1**, lines can be displayed by line number.

Usage

ad1 [-v] [-y] [-x] [-T] [filename]

The **-v** option prints the version number and exits.

The `-y` option enables yacc/Bison debugging output, not generally useful.

The `-x` option enables lex/Flex debugging output, not generally useful.

The `-T` option starts in test mode. No connection to a running pidp-1 is made, an in-memory empty state container is used instead. An attempt is made to load the test memory with the last saved actual pidp-1 coremem save file from the `/opt/pidp1-mods` directory.

If no filename is given, then no symbols or source will be available until a file is specified by the *file* command.

Otherwise the filename can be one of:

- filename
- filename.am1
- filename.lst
- filename.sym

The three am1 filenames, *.am1*, *.lst*, and *.sym* will be derived from whichever of the above is given.

If a filename.sym file is found, then the symbols will be available for use.

If a filename.lst file is found, then the source list operations will be available for use.

If there is no .lst file but there is a .am1 file, then source can be listed by line number only.

For full functionality, programs should be assembled using the **am1** `-d` flag.

Note that the *shared* option must be on in the pidp-1 configuration file in order to be useful.

Building it

As usual, just type 'make'. This assumes you have installed the pidp1-mods release and let it install Flex and Bison.

Line numbers, addresses, and symbols

What is available depends upon which of the filenames mentioned above are present.

If there is a listing file, then a mapping between line numbers *in that listing file* and memory locations shown in the file is created.

For example, this line from a listing file:

```
11: 000106 600104 jmp loop
```

says that at line 11 in the *original* source file the instruction is at memory location 000106. But, when a listing file is available, lines from it are what will be displayed, and the line number does not necessarily match line 11, especially if there are include files.

When a listing file is used, the line number shown is ignored and the actual line number in the listing file is used for creating the map and, if a symbol file is also being used, to set the lines for the symbols.

If only a source file is being used, then the line numbers will be from that file and if a symbol file is also being used, the line numbers from the symbol file will be used for the symbol locations.

Numbers, ones complement, twos complement, oh my

The PDP-1 used 1's complement math, which no modern computers use. But, **ad1** runs on a modern computer and uses 2's complement math. It automatically converts the results of math operations (+, -, *, /, %, and unary minus) to the 1's complement representation after computation, the values that are stored by the *set* command will be the correct 1's complement values.

Remember that one annoying thing about 1's complement is that there are two values for zero, 0 and -0. 0 is all bits off, -0 is all bits on. The PDP-1 automatically converted -0 to 0 for the *add*, *sub*, *mul*, *div*, *idx*, and *isp* instructions. This is also done when values are computed in **ad1** as noted above.

Logical operations, *&*, *|*, *^*, *~*, are not math operations and do not do zero or 1's complement adjusting.

When values are displayed, they are shown as positive values, just as you would see in a program listing. The PDP-1 front panel didn't show signed digits, it just showed binary light patterns.

The one exception is the special *c* formatting directive for the *show* command, see below.

Addresses and extended memory

Because **ad1** can debug code in any memory bank, addresses are always full 16-bit values. This can be confusing because of the way the PDP-1 treated addressing within an extended memory bank. Regardless of the bank being executed in, addresses other than indirect addresses were treated as 12-bit addresses in the current bank.

For example, a *jmp 123* in bank 1 is exactly the same code as a *jmp 123* in bank 0. But, the actual address in bank 1 is 010123. This normally won't cause a problem, but there are two cases where addressing is important.

First, breakpoints are set on full addresses. If you want a breakpoint on address 100 in bank 1, you must use the full value of 010123.

Second, symbols with the same name can exist in different banks with different values. **ad1** keeps symbols with their full address, provided in the symbol table produced by **am1**.

An example, assume a location in bank 1 named *loc*, and a location in bank 0 also named *loc*. Which one is the one to use?

This is handled in two ways.

First, a default current bank can be set with the *bank* command. All unqualified symbol lookups will then only match symbols in that bank.

Second, a *bank qualifier* can be added to a symbol name, e.g., *loc,1* to explicitly identify the bank.

Expressions

In general, anywhere a numeric value is needed, an expression can be used, e.g:

```
start 100
start foo
start foo+1
show .+5
```

An expression is composed of numbers, symbol names, special symbols and math operators. A regular symbol is the same as used in the assembler, a *location assignment*. The value of a symbol is the memory address where storage for it is allocated.

The special symbol `.` (period, dot), means *the last address used*, similar to its use in an assembler.

Operators

The following operators are defined and listed in order of increasing priority.

The priority is the same as that for `C` and the operations are the same.

Operator	Meaning
	bitwise or
^	bitwise xor
&	bitwise and
<< >>	left-shift right-shift
+ -	addition subtraction
* /	multiply divide
~	bitwise complement
-	unary minus, -n
()	expression nesting

Registers

Several commands can reference registers in the `pidp-1`. The register names are:

- `ac` the AC register
- `io` the IO register
- `pc` the PC register
- `ma` the memory address register
- `mb` the memory buffer register
- `as` the address switches including the extended address
- `tw` the test word switches
- `pf`, `pf1-6` the program flags
- `ss` the sense switches

Of these, the `ac`, `io`, `pc`, and `pf` can be modified via the `set` command.

Using pf refers to all 6 program flags, pf1-6 to each individual flag.

The commands

The general form of a command is:

```
command
command [optional arguments]
command arguments [optional arguments]
```

Commands can generally be shortened, the number of characters required depends upon the command. There is built-in help for the commands, and it will show the minimum length required.

For the commands below, all are lower-case. The upper case letters indicate the minimum length for the command.

Arguments are in several forms. The names used in the descriptions means:

- integer, an integer number
- value, an integer , or,
- value, a symbol imported from an **am1** symbol file, with an optional *bank qualifier, symbol, banknum*
- expression, as noted above
- special symbol, as noted in the command description
- address, a positive value or expression within the memory bounds of the pidp-1, 16 banks of 4096 words.

Help [command-name or topic]

With no argument, a list of all commands and topics is shown with the minimum length needed indicated.

If a command name or topic is given, the specific help for that is shown.

BASe integer

Sets the default base that numbers will use when typed in.

The valid choices are 2, 8, 10, and 16.

This is overridden by an explicit base in the number.

The initial base is 8.

BAnk integer

Sets the default memory bank in use, 0-15 in decimal.

When set to any value other than zero, all addresses that are < 4096 in decimal are assumed to be in this bank.

A base qualifier, *val, basenum*, overrides this setting.

Break address [count]

Sets a breakpoint at the given address with an optional count of how many times it must be hit before it is reported.

The breakpoint is automatically enabled.

When reported, the pidp-1 is halted.\

Breakpoints are based on the address in the PC, so breakpoints placed in memory not referenced by the PC will never be hit, such as data or variables. Use a watch instead.

Continue

This is equivalent to using the continue switch on the front panel. If the pidp-1 is halted, execution is resumed.

DElete [Watch] [integer]

If the integer is the number of a set breakpoint, delete it.

If no integer is given, a prompt is given and if confirmed deletes all breakpoints.

If watch is specified, then this applies to watches instead of breakpoints. It can be shortened the same as when used as a command.

DIstable [Watch] [integer]

If the integer is the number of a set breakpoint, disable but don't delete it.

It will not be procoessed if encountered until it is enabled again.

If watch is specified, then this applies to watches instead of breakpoints. It can be shortened the same as when used as a command.

File name

Open a new file for use with **ad1**.

The current file, if any, is first closed. Any symbols and line maps are cleared.

The name is entered without any quotes.

The name can contain a file path.

If it can't be found an error will be reported.

The name can be the basename of a program file, or have a *.am1*, *.sym*, or a *.lst* extension, the proper names will be derived. This is the same as if the name was given on the command line when **ad1** was started.

Enable [Watch] [integer]

If the integer is the number of a set breakpoint, enable it if it is enabled.

It will again be procoessed if encountered.

If watch is specified, then this applies to watches instead of breakpoints. It can be shortened the same as when used as a command.

List [number | symbol | @number| .]

If a source or listing file is open, list lines of text from it.

If no argument is given, list from the next line after the last one listed, or if none, the first line.

If the argument is a number, it is the line number to list.

If it is a symbol, it is the line number of the line the symbol was assigned a location.

If the number is preceded by @, then it is the line that corresponds to that address in the listing file, *line at address*.

if a . (period, dot) is given, it means *list from the line corresponding to the last address used*.

If an empty line is entered, it is equivalent to enterint this command with no arguments.

If the file was just a source file, then lines can only be viewed by line number.

Otherwise, if the **ad1** .lst file is found, full functionality is available.

Next

Repeat the last *show* command at the next address location. This does not change the pidp-1's PC, it is **ad1**'s local address location.

Quit

Any breakpoints and watches are deleted and **ad1** exits. The pidp-1 is left in whatever state it is in.

SEt address|register value

The location in memory or the given register is set to the value.

If the value is a negative number, it is adjusted to the 1's complement equivalent and masked to 18 bits.

If a register is given, it is similarly set, but if the register is a program flag, it can only be set to 1 or 0.

See the section on *registers* for more details.

This is roughly equivalent to using the deposit switch on the front panel.

SHow address|register [format]

Shows the contents of a memory address or register with an optional format modifier.

For an address, this is roughly equivalent to using the examine switch on the pidp-1 front panel.

See the section on *registers* for more details on registers.

An optional format can be given, and is one of:

- b show the value in binary
- o show the value in octal
- d show the value in decimal
- x show the value in hexadecimal
- c treat the value as a 1's complement 12 bit number, print the equivalent negative number if the value is negative
- a show the value as 2 ascii characters, the first in the high 9 bits, the second in the low 9 bits

- *f* show the value as 3 flex characters, packed as 6 bits each

There are two pseudo-registers, *break* and *watch*, which can be shortened in the same way as the command name. These will list all set breakpoints or watches and their status.

Displaying the *break*, *watch*, *pf*, or *ss* registers ignores any optional format. The *pf* and the *ss* registers are always displayed in binary.

The *a* and *f* formats use the same value formats as the **ad1** '*c*', *ascii*, *flex*, and *text* instructions produce.

This is (very) roughly equivalent to using the examine switch on the front panel.

STart address

This is equivalent to entering a start address on the pidp-1 front panel and using the start switch. The address is a full 16-bit address, so execution can start in any bank.

Step

This is equivalent to using the continue switch on the front panel while the single-instruction switch is on. One instruction cycle is executed.

STOp

This is equivalent to using the stop switch on the front panel, the pidp-1 is halted.

Watch address [value]

Sets a watch at the given address with an optional value to watch for.

If a value is given, a change of the value in memory to this value signals the watch.

If no value is given, then any change of the value in memory signals the watch.

The watch is automatically enabled.

Rewriting the same value into memory as was there does not count as a change.

When reported, the pidp-1 is halted.

Watches are based on the contents of the address in memory and thus can be used on any memory location that is modified, including instructions that are modified.

The address is a full 16-bit address.